Session 12: Multi-omics Data Analysis, Part II.

Hokeun Sun

Associate Professor
Department of Statistics
Pusan National University

Contents

1. Statistical Variable Selection and Cross-validation
2. Analysis of High-dimensional Genomic Data I
   - Lasso
   - Elastic-net
3. Analysis of High-dimensional Genomic Data II
   - Group lasso
   - Sparse group lasso
   - Network-based regularization
4. Analysis of High-dimensional DNA Methylation Data

1. **Statistical Variable Selection and Cross-validation**

2. Analysis of High-dimensional Genomic Data I
3. Analysis of High-dimensional Genomic Data II
4. Analysis of High-dimensional DNA Methylation Data

Statistical Learning

- Statistical learning is a set of tools for modeling and understanding complex datasets.
- It is recently developed area in statistics along with machine learning in computer science.
- With the explosion of "Big Data" problems, statistical learning has become a very hot field in many scientific areas.
- The tools of statistical learning can be classified as supervised or unsupervised learning
- Supervised statistical learning builds a statistical model for predicting or estimating for data with an output based on one or more inputs.
- Unsupervised statistical learning learns relationships and structures from data that has inputs but no supervising output.

The Supervised Learning Problem

- Outcome measurement Y (also called dependent variable, response, target, phenotype outcome).
- Vector of p predictor measurements X (also called inputs, regressors).
  - Genomic features (SNP, gene exparession levels, ...)
  - Clinical/environmental features (age, gender, ...)
- In the regression problem, Y is quantitative (e.g numerical values of trait, blood pressure, cholesterol levels, ...)
- In the classification problem, Y takes values in a finite, unordered set (e.g., survived/died, cancer class of tissue sample, case/control, ...)
- We have training data $(x_1, y_1), \ldots, (x_n, y_n)$. These are observations (examples, instances) of these measurements.

Supervised Learning: Model

- Now we write our model as

$$Y = f(X) + \epsilon$$

  where ϵ captures measurement errors and other discrepancies.
- With a good $f(\cdot)$ we can make predictions of Y at new points $X = x$.
- Depending on the complexity of $f(\cdot)$, we may be able to understand how each component X_j of X affects Y.
- The function f that connects the input variable to the output variable is in general unknown.
- Statistical learning refers to a set of approaches for estimating the function f.

Parametric and Non-parametric Methods

- Observations we have are called the training data because we use these observations to train or teach our method.
- Our goal is to apply a statistical learning method to the training data in order to estimate the unknown function f.
- We want to find a function $\hat{f}$ such that

$$Y \approx \hat{f}(X)$$

  for any observation (X, Y).
- Most statistical learning methods for this task can be characterized as either parametric or non-parametric.
- One of representative parametric approaches is linear regression.

Linear Model

- The linear regression is defined as

$$Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p + \epsilon$$

- Linear regression is assumes that the dependence of Y on X_1, X_2, $\cdots$, X_p is linear.
- True regression functions are never linear!
- Although it may seem overly simplistic, linear regression is extremely useful both conceptually and practically.
- The linear model has distinct advantages in terms of its interpretability and often shows good predictive performance.

Ordinary Least Square Estimate

- In linear regression, regression coefficients,

$$\boldsymbol{\beta} = \{\beta_0, \ldots, \beta_p\}$$

  are unknown and need to be estimated.
- The ordinary least square (OLS) estimate for $\boldsymbol{\beta}$ minimizes the residual sum of squares. i.e.,

$$\hat{\boldsymbol{\beta}}^{OLS} = \underset{\boldsymbol{\beta} \in \mathbb{R}^{p+1}}{\arg\min}\, \mathsf{RSS}(\boldsymbol{\beta})$$

  where

$$\mathsf{RSS}(\boldsymbol{\beta}) = \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2$$

Ordinary Least Square Estimate

- Gauss-Markov Theorem: OLS estimates for $\boldsymbol{\beta}$ have the smallest variance among all linear unbiased estimates.
- Problems in multiple linear regression
  - Some coefficients in $\boldsymbol{\beta}$ are not well estimated / have large variance for correlated predictors, since correlated variables carry the same information regarding the response.
  - OLS cannot be computed when $n < p$ (high-dimensional data).
- Alternative approach
  - Prediction accuracy can be improved by setting some coefficients to 0.
  - For interpretation, we need to determine a smaller subset that exhibits the strongest effects on the response from a large number of predictors.

Three Classes of Methods

- Subset Selection: We identify a subset of the p predictors that we believe to be related to the response. We then fit a model using OLS on the reduced set of variables.
- Shrinkage: We fit a model involving all p predictors, but the estimated coefficients are shrunken towards zero relative to the OLS estimates. This shrinkage (also known as regularization) has the effect of reducing variance and can also perform variable selection.
- Dimension Reduction: We project the p predictors into a M-dimensional subspace, where $M < p$. This is achieved by computing M different linear combinations, or projections, of the variables. Then, these M projections are used as predictors to fit a linear regression model by OLS.

Shrinkage Methods

- We can fit a model containing all p predictors using a technique that constrains or regularizes the coefficient estimates, or equivalently, that shrinks the coefficient estimates towards zero.
- It turns out that shrinking the coefficient estimates can significantly reduce their variance.
- Lasso (Least absolute shrinkage and selection operator)
- The lasso coefficients, $\hat{\boldsymbol{\beta}}^{lasso}$, minimize the quantity

$$\text{RSS}(\boldsymbol{\beta}) + \lambda \|\boldsymbol{\beta}\|_1 = \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^{p} |\beta_j|,$$

where $\lambda > 0$ is a tuning parameter to control sparsity.

Lasso

- The l_1 penalty of the lasso has the effect of forcing some of the coefficient estimates to be exactly equal to zero when the tuning parameter λ is sufficiently large.
- Hence, the lasso performs variable selection.
- The lasso yields sparse models - that is, models that involve only a subset of the variables.
- The tuning parameter λ controls the number of nonzero regression coefficients (degrees of freedom).
- Selecting an appropriate value of λ for the lasso is crucial.
- Alternatively, selection probability of individual variables can be computed without selection of the optimal tuning parameter.

Lasso

- For a grid of λ values such as

$$\lambda_{\text{max}} = \lambda_1 > \lambda_2 > \ldots > \lambda_{m-1} > \lambda_m = \lambda_{\text{min}},$$

  the number of nonzero regression coefficients (degrees of freedom: df) is

$$df(\hat{\boldsymbol{\beta}}_{\lambda_1}) = 0 \leq df(\hat{\boldsymbol{\beta}}_{\lambda_2}) \leq \ldots \leq df(\hat{\boldsymbol{\beta}}_{\lambda_{m-1}}) \leq df(\hat{\boldsymbol{\beta}}_{\lambda_m})$$

- The optimal value of λ should be chosen by a resampling method such as cross-validation.
- Numerical values of nonzero regression coefficients also rely on λ.

Example: Heart Data

- These data contains a binary outcome `AHD` for 303 patients who presented with chest pain.
- An outcome value of `Yes` indicates the presence of heart disease based on an angiographic test, while No means no heart disease.
- There are 13 predictors including `Age`, `Sex`, `Chol` (a cholesterol measurement), and other heart and lung function measurements.
- We can find some missing values from data, so we removed the samples with missing values. Then, the sample size is reduced down to 297.

Example: Heart Data

```
## Visit my github website to download some packages
## https://github.com/statsun78/kogo
## library(kogo)

data(Heart)
str(Heart)
head(Heart)
summary(Heart)

g0 <- glm(AHD ~., family="binomial", Heart)
summary(g0)

## library(glmnet)
x <- model.matrix(AHD ~., Heart)
y <- Heart$AHD
g1 <- glmnet(x, y, family="binomial")

par(mfrow=c(1,2))
plot(g1, "lambda", label=TRUE)
plot(g1, "norm", label=TRUE)
```

Example: Heart Data

```
names(g1)
g1$df
g1$lambda
dim(g1$beta)

df <- cbind(g1$lambda, g1$df)
colnames(df) <- c("lambda", "nonzero")
rownames(df) <- colnames(g1$beta)
df

g1$beta[, c("s0", "s5", "s15", "s25", "s35")]

s68 <- as.numeric(c(g1$a0["s68"], g1$beta[-1,"s68"]))
s20 <- as.numeric(c(g1$a0["s20"], g1$beta[-1,"s20"]))
s10 <- as.numeric(c(g1$a0["s10"], g1$beta[-1,"s10"]))
s2 <- as.numeric(c(g1$a0["s2"], g1$beta[-1,"s2"]))
data.frame(OLS=g0$coef, lasso.s68=s68, lasso.s20=s20,
           lasso.s10=s10, lasso.s2=s2)
```

Training Set

- The goal of supervised learning tasks is to use the training set and test set to minimize the test error.
- Training set is used to train the model, i.e., to estimate the parameters such as regression coefficients. Suppose that we have m different models

$$\hat{f}_{\lambda_1}(x), \hat{f}_{\lambda_2}(x), \hat{f}_{\lambda_3}(x), \ldots, \hat{f}_{\lambda_m}(x)$$

- The l-th model has $\hat{\boldsymbol{\beta}}_l = \{\hat{\beta}_{0,l}, \hat{\beta}_{1,l}, \ldots, \hat{\beta}_{p,l}\}$ such that

$$\hat{f}_{\lambda_l}(x) = \hat{\beta}_{0,l} + \hat{\beta}_{1,l}x_1 + \cdots + \hat{\beta}_{p,l}x_p$$

- $\hat{\boldsymbol{\beta}}_l$ should be estimated based only on training set.

Test Set

- Test set is used to assess the model performance. The test set data should not used in the model building process.
- Given the test data $T = \{x_i, y_i\}$ for $i \in \{1, \ldots, N\}$, the mean squared error (MSE) of λ_l is

$$MSE_{\lambda_l} = \frac{1}{N} \sum_{i \in T} \left(y_i - \hat{f}_{\lambda_l}(x_i) \right)^2$$

- We can find the best model among m models as comparing the test set error of m models.

$$\hat{\lambda} = \underset{\lambda_1, \ldots, \lambda_m}{\arg\min} MSE_{\lambda_l}$$

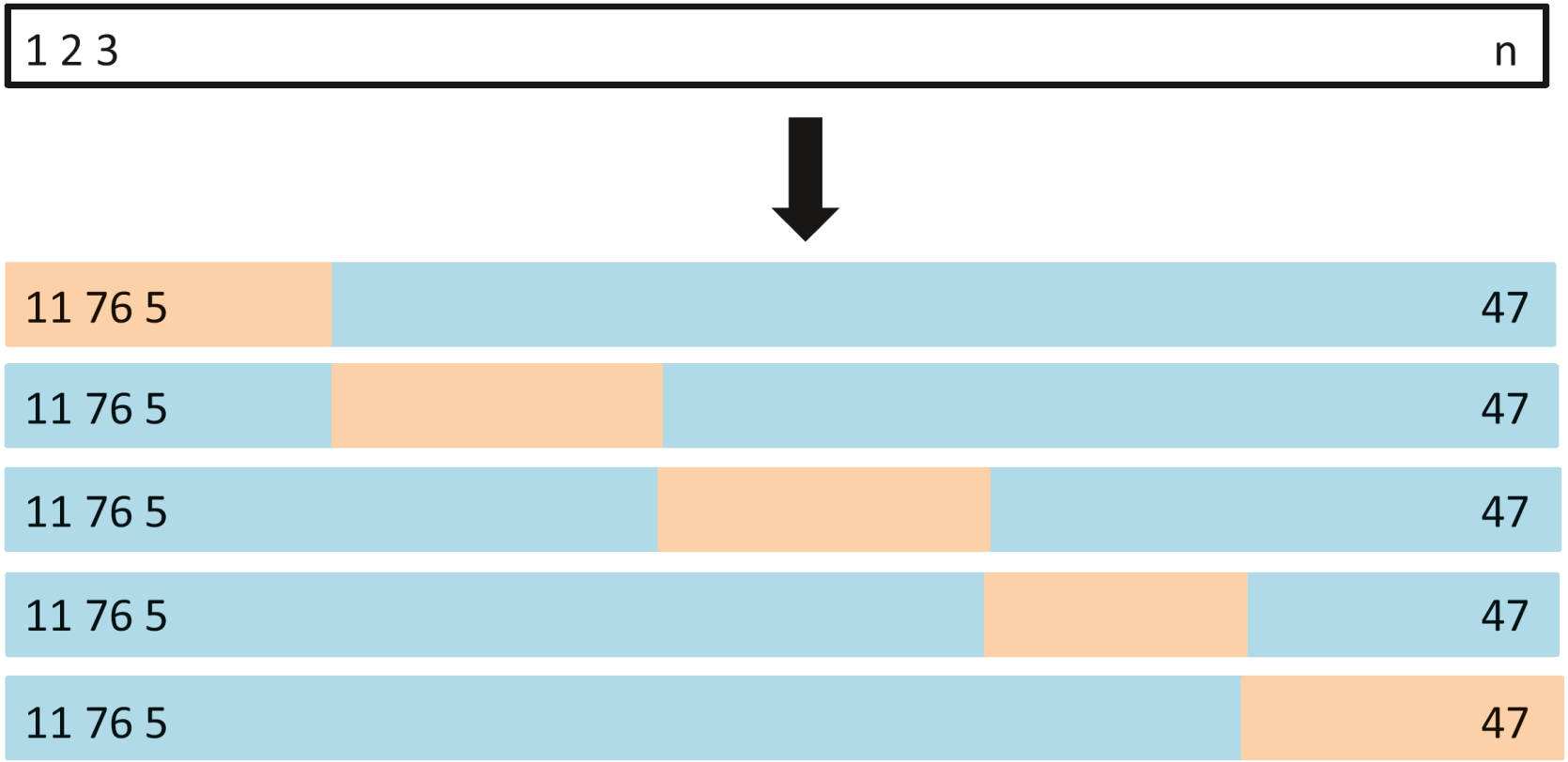
- Test set is often called validation set in tuning parameter selection.

K-fold Cross-validation

- In order to compute the test error, we need to have a large designated test set. However, it is not often available.
- The K-fold cross-validation (CV) is the most popular approach to estimate the test error.
- For each model,
  1. Randomly divide the data into K equal-sized parts.
  2. Leave out part k, and estimate the regression coefficients based on the other $K-1$ parts (training set).
  3. Compute MSE for the left-out kth part (test set).
  4. This is done in turn for each part $k = 1, 2, \cdots, K$.
  5. Final MSE value is then averaged over K parts.

K-fold Cross-validation

- Divide data into K roughly equal-sized parts ($K = 5$ here)



K-fold CV for Lasso

- Suppose that y_i is a quantitative value of the i-th individual.
- For lasso, the K-fold CV procedure:
  1. Randomly separate n samples into K folds: $C_1, C_2, \ldots, C_K$
  2. For each λ, compute regression coefficients based on C_{-k},

     $$\hat{\boldsymbol{\beta}}_{\lambda_1}^{[-k]}, \hat{\boldsymbol{\beta}}_{\lambda_2}^{[-k]}, \ldots, \hat{\boldsymbol{\beta}}_{\lambda_{m-1}}^{[-k]}, \hat{\boldsymbol{\beta}}_{\lambda_m}^{[-k]}$$

     Note that C_{-k} is an observation set with part k removed.
  3. Compute cross-validation error (CVE) for each λ.

     $$\text{CVE}(\lambda_l) = \frac{1}{n} \sum_{k=1}^{K} \sum_{i \in C_k} \left(y_i - x_i^{\mathrm{T}} \hat{\boldsymbol{\beta}}_{\lambda_l}^{[-k]} \right)^2$$

     for $l = 1, 2, \ldots, m$.
  4. Pick up the optimal $\hat{\lambda}_l$ that minimizes $\text{CVE}(\hat{\lambda}_l)$.

CV Error for Binary Outcomes

- Suppose that $y_i = 1$ for a case and $y_i = 0$ for a control.
- CVE based on deviance

$$\text{CVE} = \frac{1}{n} \sum_{k=1}^{K} -2 \sum_{i \in C_k} \left(y_i \log \hat{p}_i^{[-k]} + (1 - y_i) \log(1 - \hat{p}_i^{[-k]}) \right),$$

where $\hat{p}_i^{[-k]}$ is the probability of $y_i = 1$ for the observation i, obtained from the data with part k removed.

- CVE based on classification error

$$\text{CVE} = \frac{1}{n} \sum_{k=1}^{K} \sum_{i \in C_k} I\left(y_i - \hat{y}_i^{[-k]} \right),$$

where $\hat{y}_i^{[-k]}$ is the fit for the observation i, obtained from the data with part k removed.

Example: Heart Data

```
set.seed(1234)
K <- 10
n <- length(y)
fold <- sample(rep(1:K, length.out=n))
lam <- g1$lambda

MSE <- matrix(0, n, length(lam))
for (i in 1:K) {
    test <- fold==i
    tran <- fold!=i
    g <- glmnet(x[tran,], y[tran], family="binomial", lambda=lam)
    prob <- predict(g, x[test,], s=lam, type="response")
    yval <- y[test]=="Yes"
    MSE[test, ] <- -2*(yval*log(prob) + (1-yval)*log(1-prob))
}
CVE <- apply(MSE, 2, mean)

plot(log(lam), CVE, type="l")
which.min(CVE)
lam[which.min(CVE)]
```

Example: Heart Data

```
set.seed(123)

R <- 8
CVE <- matrix(0, R, length(lam))

for (j in 1:R) {
    fold <- sample(rep(1:K, length.out=n))
    MSE <- matrix(0, n, length(lam))
    for (i in 1:K) {
        test <- fold==i
        tran <- fold!=i
        g <- glmnet(x[tran,], y[tran], family="binomial", lambda=lam)
        prob <- predict(g, x[test,], s=lam, type="response")
        yval <- y[test]=="Yes"
        MSE[test, ] <- -2*(yval*log(prob) + (1-yval)*log(1-prob))
    }
CVE[j,] <- apply(MSE, 2, mean)
}
```

Example: Heart Data

```
matplot(log(lam), t(CVE), type="l", xlab=expression(log(lambda)),
      ylab="Deviance", main="10-fold CV")

ww <- apply(CVE, 1, which.min)
ww
for (k in 1:length(ww)) {
    abline(v=log(lam)[ww[k]], col="red", lty=2)
}

w1 <- sort(unique(ww))
beta <- g1$beta[, w1]
beta[1,] <- g1$a0[w1]
beta

data.frame(OLS=g0$coef, s35=as.numeric(beta[,1]),
           s40=as.numeric(beta[,5]))
```

1. Statistical Variable Selection and Cross-validation

2. Analysis of High-dimensional Genomic Data I

3. Analysis of High-dimensional Genomic Data II
4. Analysis of High-dimensional DNA Methylation Data

Genetic Association Studies

- In genetic association studies, the goal is to identify SNP, genetic variants, genes or gene sets that are associated with diseases/traits.
- It is a statistically challenging problem due to
  - Problem of dimensionality, i.e., $p \gg n$
  - Highly correlated genomic data (LD)
  - Considering covariate effects (non-genetic effects)
  - Sparsity of true signals
- Univariate analysis where each variable (a group or variables) is tested one at a time cannot be accounted for correlation among genetic variables.
- Multiple linear regression cannot be applied because high variance of $\hat{\boldsymbol{\beta}}$ and $p \gg n$.
- Alternatives: regularization methods (penalized regression)!

Penalized Regressions in Recent Bioinformatics

BIOINFORMATICS ORIGINAL PAPER Vol. 27 no. 24 2011, pages 3399–3406
doi:10.1093/bioinformatics/btr591

Gene expression

Optimized application of penalized regression methods to diverse genomic data

Levi Waldron[1,3], Melania Pintilie[2], Ming-Sound Tsao[3], Frances A. Shepherd[3], Curtis Huttenhower[1,*] and Igor Jurisica[3,*]

BIOINFORMATICS ORIGINAL PAPER Vol. 25 no. 6 2009, pages 714–721
doi:10.1093/bioinformatics/btp041

Genome analysis

Genome-wide association analysis by lasso penalized logistic regression

Tong Tong Wu[1], Yi Fang Chen[2], Trevor Hastie[2,3], Eric Sobel[4] and Kenneth Lange[4,5,*]

BIOINFORMATICS ORIGINAL PAPER Vol. 27 no. 19 2011, pages 2679–2685
doi:10.1093/bioinformatics/btr450

Systems biology

Advance Access publication July 30, 2011

Penalized regression elucidates aberration hotspots mediating subtype-specific transcriptional responses in breast cancer

Yinyin Yuan[1,2,*], Oscar M. Rueda[1,2], Christina Curtis[1,2,†] and Florian Markowetz[1,2,*]

BMC Bioinformatics

RESEARCH ARTICLE Open Access

Sparse logistic regression with a $L_{1/2}$ penalty for gene selection in cancer classification

Yong Liang[1*], Cheng Liu[1], Xin-Ze Luan[1], Kwong-Sak Leung[2], Tak-Ming Chan[2], Zong-Ben Xu[3] and Hai Zhang[3]

Penalized Regressions in Recent Bioinformatics

Bioinformatics, 33(12), 2017, 1765–1772
doi: 10.1093/bioinformatics/btx064
Advance Access Publication Date: 6 February 2017
Original Paper

OXFORD

Genome analysis

pETM: a penalized Exponential Tilt Model for analysis of correlated high-dimensional DNA methylation data

Hokeun Sun[1], Ya Wang[2], Yong Chen[3], Yun Li[4,5,6] and Shuang Wang[2,*]

[1]Department of Statistics, Pusan National University, Busan 609-735, Korea, [2]Department of Biostatistics, Mailman School of Public Health, Columbia University, New York, NY 10032, USA, [3]Division of Biostatistics, Perelman School of Medicine, University of Pennsylvania, Philadelphia, PA 19104, USA, [4]Department of Biostatistics, [5]Department of Genetics and [6]Department of Computer Science, University of North Carolina, Chapel Hill, NC 27599, USA

*To whom correspondence should be addressed.
Associate Editor: John Hancock

Journal of Bioinformatics and Computational Biology
Vol. 16, No. 4 (2018) 1850010 (18 pages)
© World Scientific Publishing Europe Ltd.
DOI: 10.1142/S0219720018500105

World Scientific
www.worldscientific.com

New variable selection strategy for analysis of high-dimensional DNA methylation data

Jiyun Choi, Kipoong Kim and Hokeun Sun*
Department of Statistics
Pusan National University
Busan 46241, Korea
**hsun@pusan.ac.kr*

Kim and Sun *BMC Bioinformatics* (2019) 20:510
https://doi.org/10.1186/s12859-019-3040-x

BMC Bioinformatics

METHODOLOGY ARTICLE Open Access

Check for updates

Incorporating genetic networks into case-control association studies with high-dimensional DNA methylation data

Kipoong Kim and Hokeun Sun*

Regularization Methods

- Regularization methods are based on a penalized likelihood

$$Q_\lambda(\boldsymbol{\beta}) = -l(\boldsymbol{\beta}) + p_\lambda(\boldsymbol{\beta})$$

- $l(\boldsymbol{\beta})$ is a log-likelihood function of $\boldsymbol{\beta} = \{\beta_1, \ldots, \beta_p\}$, which is the p-dimensional vector of regression coefficients.
- $l(\boldsymbol{\beta})$ is determined by different types of phenotype outcomes.
- $p_\lambda(\boldsymbol{\beta})$ is a penalty function, where a tuning parameter λ controls sparsity.
- Solution to regression coefficient $\hat{\boldsymbol{\beta}}$ can be obtained by

$$\hat{\boldsymbol{\beta}}_\lambda = \arg\min_{\boldsymbol{\beta}} Q_\lambda(\boldsymbol{\beta}),$$

for a fixed value of λ.

Penalized Likelihood for Quantitative Outcome

- For the i-th individual,
  - y_i : a quantitative outcome
  - $x_i = (x_{i1}, \ldots, x_{ip})^{\mathrm{T}}$: the p-dimensional genomic data.
  - $z_i = (z_{i1}, \ldots, z_{iq})^{\mathrm{T}}$: the q-dimensional covariates.
- Linear regression model can be defined as

$$y_i = \beta_0 + z_i^{\mathrm{T}} \boldsymbol{\alpha} + x_i^{\mathrm{T}} \boldsymbol{\beta} + \epsilon_i,$$

  where $\boldsymbol{\alpha} = \{\alpha_1, \ldots, \alpha_q\}^{\mathrm{T}}$ and $\epsilon \sim N(0, \sigma^2)$.
- Least squared loss is used for the negative log-likelihood

$$\begin{aligned} Q_\lambda(\beta_0, \boldsymbol{\alpha}, \boldsymbol{\beta}) = & -l(\beta_0, \boldsymbol{\alpha}, \boldsymbol{\beta}) + p_\lambda(\boldsymbol{\beta}) \\ = & \frac{1}{2} \sum_{i=1}^{n} \left(y_i - \beta_0 - z_i^{\mathrm{T}} \boldsymbol{\alpha} - x_i^{\mathrm{T}} \boldsymbol{\beta}\right)^2 + p_\lambda(\boldsymbol{\beta}) \end{aligned}$$

Penalized Likelihood for Binary Outcome

- For the i-th individual,
  - y_i : a binary outcome ($y_i = 1$ for cases and $y_i = 0$ for controls)
- In case-control studies, the penalized likelihood is

$$Q_\lambda(\beta_0, \boldsymbol{\alpha}, \boldsymbol{\beta}) = -l(\beta_0, \boldsymbol{\alpha}, \boldsymbol{\beta}) + p_\lambda(\boldsymbol{\beta}).$$

- A logistic likelihood $l(\beta_0, \boldsymbol{\alpha}, \boldsymbol{\beta})$ is

$$= \sum_{i=1}^{n} \left[y_i \log p_i(\beta_0, \boldsymbol{\alpha}, \boldsymbol{\beta}) + (1 - y_i) \log(1 - p_i(\beta_0, \boldsymbol{\alpha}, \boldsymbol{\beta})) \right],$$

where

$$p_i(\beta_0, \boldsymbol{\alpha}, \boldsymbol{\beta}) = \frac{e^{\beta_0 + z_i^{\mathrm{T}} \boldsymbol{\alpha} + x_i^{\mathrm{T}} \boldsymbol{\beta}}}{1 + e^{\beta_0 + z_i^{\mathrm{T}} \boldsymbol{\alpha} + x_i^{\mathrm{T}} \boldsymbol{\beta}}}.$$

Penalized Likelihood for Matched Case-Control Outcome

- For the i-th individual,
  - y_i : a binary outcome ($y_i = 1$ for cases and $y_i = 0$ for controls)
  - $\delta_i \in \{1, \ldots, K\}$: the stratum of the i-th individual
- For $1 : m$ matched case-control study, the penalized likelihood

$$Q_{\boldsymbol{\alpha},\lambda}(\boldsymbol{\beta}) = -\log L(\boldsymbol{\alpha}, \boldsymbol{\beta} | n_1, \ldots, n_K) + p_\lambda(\boldsymbol{\beta}).$$

- A conditional logistic likelihood is defined as

$$L(\boldsymbol{\alpha}, \boldsymbol{\beta} | n_1, \ldots, n_K) = \prod_{k=1}^{K} \frac{\exp(\sum_{i \in \Delta_k} z_i^{\mathrm{T}} \boldsymbol{\alpha} + x_i^{\mathrm{T}} \boldsymbol{\beta} y_i)}{\sum_{i \in \Delta_k} \exp(z_i^{\mathrm{T}} \boldsymbol{\alpha} + x_i^{\mathrm{T}} \boldsymbol{\beta})},$$

where $\Delta_k = \{i \leq n : \delta_i = k\}$ consists of $m + 1$ indices.

Penalized Likelihood for Count Outcome

- For the i-th individual,
  - y_i : a count outcome ($y_i = 0, 1, 2, \ldots$)
- In Poisson regression model, the penalized likelihood is

$$Q_\lambda(\beta_0, \boldsymbol{\alpha}, \boldsymbol{\beta}) = -l(\beta_0, \boldsymbol{\alpha}, \boldsymbol{\beta}) + p_\lambda(\boldsymbol{\beta}).$$

- A Poisson likelihood is defined as

$$l(\beta_0, \boldsymbol{\alpha}, \boldsymbol{\beta}) = \sum_{i=1}^{n} \left[y_i \left(\log \mu_i(\beta_0, \boldsymbol{\alpha}, \boldsymbol{\beta}) \right) - \mu_i(\beta_0, \boldsymbol{\alpha}, \boldsymbol{\beta}) \right]$$

where

$$\mu_i(\beta_0, \boldsymbol{\alpha}, \boldsymbol{\beta}) = e^{\beta_0 + z_i^{\mathrm{T}} \boldsymbol{\alpha} + x_i^{\mathrm{T}} \boldsymbol{\beta}}.$$

Penalized Likelihood for Censored Survival Outcomes

- For the i-th individual,
  - t_i : Either survival or censoring time of the i-th individual
  - δ_i : the indicator of failure ($\delta_i = 1$) or censoring ($\delta_i = 0$)
- The penalized likelihood is

$$Q_\lambda(\boldsymbol{\alpha}, \boldsymbol{\beta}) = -\log L(\boldsymbol{\alpha}, \boldsymbol{\beta}) + p_\lambda(\boldsymbol{\beta}).$$

- A Cox partial likelihood can be written as

$$L(\boldsymbol{\alpha}, \boldsymbol{\beta}) = \prod_{i=1}^{n} \left\{ \frac{\exp(z_i^{\mathrm{T}} \boldsymbol{\alpha} + x_i^{\mathrm{T}} \boldsymbol{\beta})}{\sum_{j \in R_i} \exp(z_i^{\mathrm{T}} \boldsymbol{\alpha} + x_j^{\mathrm{T}} \boldsymbol{\beta})} \right\}^{\delta_i},$$

where R_i is the index set for observations that are at risk before time t_i.

Types of Penalty Functions

- Convex penalty functions
  - Lasso (Tibshirani, JRSS, 1996)
  - Fused lasso (Tibshirani *et al.* JRSS, 2005)
  - Adaptive lasso (Zou, JASA, 2006)
  - Elastic-net (Zou and Hastie, JRSS, 2005)
- Non-convex penalty functions
  - l_q-norm penalty with $0 < q < 1$
  - Smoothly clipped absolute deviation (SCAD) (Fan and Li, JASA, 2005)
  - Minimax concave penalty (MCP) (Zhang, AOS, 2010)
- Group structure penalty functions
  - Group lasso (Yuan and Lin, JRSS, 2006)
  - Graph-constrained regularization (Li and Li, AOAS 2010)
  - Sparse group lasso (Simon *et al.* JGCS 2013)

Lasso Regression and Ridge Regression

- Lasso regression uses a l_1-norm penalty

$$p_\lambda(\boldsymbol{\beta}) = \lambda \|\boldsymbol{\beta}\|_1 = \lambda \sum_{j=1}^{p} |\beta_j|.$$

- Lasso can perform variable selection since
  - $\hat{\beta}_j = 0$ for most of $j \in \{1, 2, \ldots, p\}$.
  - $\hat{\beta} \neq 0$ for only a few j.
- Ridge regression uses a squared l_2-norm penalty

$$p_\lambda(\boldsymbol{\beta}) = \lambda \|\boldsymbol{\beta}\|_2^2 = \lambda \sum_{j=1}^{p} \beta_j^2$$

Lasso Regression vs. Ridge Regression

- Variable selection
  - Ridge regression shrinks the coefficient estimates towards zero, just like lasso.
  - But, ridge regression coefficients are not exactly equal to zero, unlike lasso.
  - Technically, ridge regression cannot perform variable selection.
- Correlated variables
  - Ridge regression is likely to shrink the coefficient estimates of correlated variables together.
  - Lasso regression often selects only one of correlated variables even though all of them are related with a response.
  - Lasso regression is not ideal for highly correlated predictors.
- Neither ridge nor lasso will universally dominate the other.

Lasso Regression with Covariates

- Suppose that we have a q-dimensional covariate vector. The covariate of the i-individual is

$$z_i = (z_{i1}, z_{i2}, \ldots, z_{iq})^{\mathrm{T}}$$

- The lasso solution $\hat{\boldsymbol{\beta}}_\lambda$ minimizes the penalized likelihood

$$\frac{1}{n}\sum_{i=1}^{n}\left(y_i - \beta_0 - \sum_{k=1}^{q}\alpha_k z_{ik} - \sum_{j=1}^{p}\beta_j x_{ij}\right)^2 + \lambda\sum_{j=1}^{p}|\beta_j|,$$

  if y_i is a quantitative outcome.
- Note that the regression coefficients of the covarates are not penalized.

One-standard-error Rule in Variable Selection

- Choose a smaller model: one-standard-error rule
- Calculate the standard error of the estimated test MSE for each λ.

$$sd\left(\mathsf{CVE}(\lambda_l)\right) = \sqrt{\frac{1}{n-1}\sum_{k=1}^{K}\left(\mathsf{MSE}_k(\lambda_l) - \frac{1}{n}\sum_{k=1}^{K}\mathsf{MSE}_k(\lambda_l)\right)^2}$$

where

$$\mathsf{MSE}_k(\lambda_l) = \sum_{i\in C_k}\left(y_i - x_i^{\mathrm{T}}\hat{\boldsymbol{\beta}}_{\lambda_l}^{[-k]}\right)^2$$

- Select the smallest model (largest λ) for which the test error is within one standard error of the lowest point on the curve.

$$\left[\min(\mathsf{CVE}) - sd\left(\mathsf{CVE}\right), \min(\mathsf{CVE}) + sd\left(\mathsf{CVE}\right)\right]$$

ALL Data

- Use `acute lymphocytic leukemia (ALL)` package from Bioconductor.
- The `ALL` data consists of microarray assays for 12,625 genes from 128 different individuals with either `B-cell` or `T-cell` acute lymphoblastic leukemia (ALL).
- 95 of the patients have `B-cell` leukemia while the other 33 have `T-cell` leukemia.
- Besides gene expression data, the `exprSet(ALL)` object also contains 21 additional covariates, including age and gender.

ALL Data

```
## library(ALL)
data(ALL)
?ALL
ALL
str(ALL)

dim(exprs(ALL))
varLabels(ALL)
ALL$sex
ALL$age
ALL$mol.biol

ALL$BT
table(ALL$BT)

BT <- ALL$BT %in% c("T", "T1","T2","T3", "T4")
y <- factor(as.numeric(BT), labels=c("B","T"))
table(y)

x <- t(exprs(ALL))
```

Lasso: ALL Data

```
xc <- cbind(sex=ALL$sex, age=ALL$age, x)
miss <- apply(xc, 1, function(t) sum(is.na(t)))
ym <- y[miss==0]
xc <- xc[miss==0,]
dim(xc)
xc[1:5, 1:10]

g0 <- glmnet(xc, ym, alpha=0, family="binomial")

par(mfrow=c(1,2))
plot(g0, "lambda")
plot(g0, "norm")

gl <- glmnet(xc, ym, alpha=1, family="binomial")

par(mfrow=c(1,2))
plot(gl, "lambda")
plot(gl, "norm")
```

Lasso: ALL Data

```
pp <- rep(1, ncol(xc))
pp[1:2] <- 0
g <- glmnet(xc, ym, alpha=1, family="binomial", penalty.factor=pp)

dim(g$beta)
g$beta[1:4,]
g$lambda
g$df
cbind(g$lambda, g$df)

plot(g, "lambda")

t <- 50
g$lambda[t]
wh <- which(g$beta[,t]!=0)
as.matrix(g$beta[wh, t])
abline(v=log(g$lambda[t]), lwd=2, lty=2, col="red")
```

Lasso: ALL Data

```
set.seed(12345)
gvc <- cv.glmnet(xc, ym, alpha=1, family="binomial",
                 penalty.factor=pp, nfolds=5)
gvc$lambda.min
gvc$lambda.1se
plot(gvc)

fit1 <- coef(gvc, s="lambda.min")
fit2 <- coef(gvc, s="lambda.1se")
c(sum(as.matrix(fit1)!=0), sum(as.matrix(fit2)!=0))

w1 <- which(as.matrix(fit1)!=0)
w2 <- which(as.matrix(fit2)!=0)

data.frame(name=colnames(xc)[w1], beta=fit1[w1])
data.frame(name=colnames(xc)[w2], beta=fit1[w2])
```

The Limitations of Lasso

- Lasso penalty function (l_1-norm)
  - If $p > n$, lasso selects at most n variables. The number of selected genes is bounded by the number of samples.
  - Lasso is indifferent to highly correlated variables and tends to pick only one variable and ignore the rest.
- Ridge penalty function (squared l_2-norm)
  - It cannot perform variable selection. (No coefficients are zero.)
  - But, ridge regression shrinks correlated features to each other
- Elastic-net regularization
  - The elastic-net combines the l_1 and squared l_2 penalties
  - The l_1 part of the penalty generates a sparse model.
  - The l_2 part of the penalty encourages grouping effects.

Elastic-Net Regularization

- The penalty function of elastic-net is

$$
\begin{aligned}
p_\lambda(\boldsymbol{\beta}) &= \lambda \left(\alpha \|\boldsymbol{\beta}\|_1 + (1-\alpha)\|\boldsymbol{\beta}\|_2^2 \right) \\
&= \lambda\alpha \sum_{j=1}^{p} |\beta_j| + \lambda(1-\alpha) \sum_{j=1}^{p} \beta_j^2
\end{aligned}
$$

  where $\alpha \in [0, 1]$ is a mixing proportion between lasso and ridge.
  - Lasso if $\alpha = 1$
  - Ridge regression if $\alpha = 0$.
- Convex penalty function
  - Efficient computation (ideal for high-dimensional data).
  - Different computational procedures lead to the same solutions.
  - Relatively many convex optimization algorithms available

Elastic-Net Regularization

- Minimize the penalized log-likelihood,

$$\hat{\boldsymbol{\beta}}_{\lambda,\alpha} = \arg\min_{\boldsymbol{\beta}} \left[-l(\boldsymbol{\beta}) + \lambda\alpha\|\boldsymbol{\beta}\|_1 + \lambda(1-\alpha)\|\boldsymbol{\beta}\|_2^2 \right],$$

  where tuning parameters α and λ are fixed.
- Cyclic coordinate descent algorithm (Friedman *et al.* 2007, 2010)
  - Efficient and fast convergence for even large p
  - Given α, the solution $\hat{\boldsymbol{\beta}}$ can be obtained for a grid of λ values,

$$\hat{\boldsymbol{\beta}}_{\lambda_1}, \hat{\boldsymbol{\beta}}_{\lambda_2}, \ldots, \hat{\boldsymbol{\beta}}_{\lambda_{m-1}}, \hat{\boldsymbol{\beta}}_{\lambda_m}$$

    for $\lambda_1 > \lambda_2 > \ldots, > \lambda_{m-1} > \lambda_m$.
- Need to pick up the optimal λ and α.

Elastic-Net: ALL Data

```
g <- glmnet(xc, ym, alpha=0.5, family="binomial", penalty.factor=pp)
g$df
g$lambda
plot(g, "lambda")

set.seed(111)
gvc <- cv.glmnet(xc, ym, alpha=0.5, family="binomial",
                 penalty.factor=pp, nfolds=5)
plot(gvc)

fit1 <- coef(gvc, s="lambda.min")
fit2 <- coef(gvc, s="lambda.1se")
c(sum(as.matrix(fit1)!=0), sum(as.matrix(fit2)!=0))

w1 <- which(as.matrix(fit1)!=0)
w2 <- which(as.matrix(fit2)!=0)
data.frame(name=colnames(xc)[w1], beta=fit1[w1])
data.frame(name=colnames(xc)[w2], beta=fit1[w2])
```

Elastic-Net: ALL Data

```
set.seed(1010)

fold1 <- sample(1:5, size=sum(ym=="B"), replace=TRUE)
fold2 <- sample(1:5, size=sum(ym=="T"), replace=TRUE)
foldid <- rep(NA, length(ym))
foldid[ym=="B"] <- fold1
foldid[ym=="T"] <- fold2

alp <- seq(0.05, 1, 0.05)
cve <- df <- NULL
for (j in 1:length(alp)) {
    g <- cv.glmnet(xc, ym, alpha=alp[j], family="binomial",
                   foldid=foldid, penalty.factor=pp)
    cve[j] <- min(g$cvm)
    df[j] <- g$nzero[g$cvm==min(g$cvm)]
}
wh <- which.min(cve)
alp[wh]
df[wh]
```

Elastic-Net: ALL Data

```
set.seed(1010)
K <- 10
alp <- seq(0.1, 1, 0.1)

res <- matrix(0, K, 3)
for (k in 1:K) {
    fold1 <- sample(1:5, size=sum(ym=="B"), replace=TRUE)
    fold2 <- sample(1:5, size=sum(ym=="T"), replace=TRUE)
    foldid <- rep(NA, length(ym))
    foldid[ym=="B"] <- fold1
    foldid[ym=="T"] <- fold2
    cve <- df1 <- df2 <- NULL
    for (j in 1:length(alp)) {
        g <- cv.glmnet(xc, ym, alpha=alp[j], family="binomial",
                       foldid=foldid, penalty.factor=pp)
        cve[j] <- min(g$cvm)
        df1[j] <- g$nzero[g$lambda==g$lambda.min]
        df2[j] <- g$nzero[g$lambda==g$lambda.1se]
    }
```

Elastic-Net: ALL Data

```
    wh <- which.min(cve)
    res[k, 1] <- alp[wh]
    res[k, 2] <- df1[wh]
    res[k, 3] <- df2[wh]
}

colnames(res) <- c("alpha", "df.min", "df.1se")
res

lam <- df <- matrix(0, 100, length(alp))
colnames(lam) <- colnames(df) <- alp
for (i in 1:length(alp)) {
    g <- glmnet(xc, ym, alpha=alp[i], family="binomial",
                penalty.factor=pp)
    lam[,i] <- g$lambda
    df[,i] <- g$df
}
apply(lam, 2, summary)
apply(df, 2, summary)
```

Selection Probability

- Stable selection of regularization methods:
  - Selection probability (Meinshausen and Bühlmann, 2010)
  - Pairs stability selection (Shah and Samworth, 2013)
- Computation of selection probability:
  - Let I_k be the k-th bootstrap samples.
  - For a fine grid of α and λ, the selection probability of the j-th gene is

$$SP_j = \max_{\alpha,\lambda} \frac{1}{K} \#\{k \leq K : \hat{\beta}_j^{\alpha,\lambda}(I_k) \neq 0\},$$

    where K is the total number of resampling.
  - $\hat{\beta}_j^{\alpha,\lambda}(I_k)$ is the estimated regression coefficient for the j-th gene based on the k-th bootstrap sample when (α, λ) is fixed.
- SP measures relative selection frequency of individual genes.
- Top ranked genes can be listed along with selection probabilities of individual genes.

Selection Probability Example

```
set.seed(10101)
lam10 <- seq(min(lam[,10]), max(lam[,10]), length.out=10)

B <- matrix(0, ncol(xc), length(lam10))
for (k in 1:100) {
    for (j in 1:length(alp)) {
        boot <- sample(1:nrow(xc), replace=TRUE)
        g <- glmnet(xc[boot,], ym[boot], alpha=alp[j], lambda=lam10,
                    family="binomial", penalty.factor=pp)
        beta <- as.matrix(g$beta)
        beta[beta!=0] <- 1
        B <- B + beta
    }
}
maxB <- apply(B, 1, max)
SP <- maxB/(100*length(alp))

top <- 12
oo <- order(SP, decreasing=TRUE)[1:top]
data.frame(sel.prob=SP[oo])
```

1. Statistical Variable Selection and Cross-validation
2. Analysis of High-dimensional Genomic Data I

3. Analysis of High-dimensional Genomic Data II

4. Analysis of High-dimensional DNA Methylation Data

High-dimensional Genomic Data with a Group Structure

- Group structured genomic data
  - Single-nucleotide polymorphism (SNP) data: genes or genetic regions includes multiple SNPs.
  - DNA methylation data: genes or genetic regions contains multiple CpG sites.
  - Gene expression data: genetic pathways or signaling pathways consist of multiple genes.
- Genomic measurements within the same group are likely to be correlated with each other.
- Penalty functions encouraging grouping effects
  - Group lasso (only group selection)
  - Sparse group lasso (both group and individual selection)
  - Network-based regularization (both group and individual selection)

Group Lasso

- Group lasso performs group selection
- Suppose that the p predictors are divided into K groups of sizes q_k with corresponding coefficients $\boldsymbol{\beta}_k$.
- Group lasso solves the convex optimization problem.

$$\hat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} \left[-l(\boldsymbol{\beta}) + \lambda \sum_{k=1}^{K} \sqrt{q_k} \|\boldsymbol{\beta}_k\|_2 \right]$$

- $\|\boldsymbol{\beta}_k\|_2 = 0$ if and only if $\boldsymbol{\beta}_k = (0, \ldots, 0)$, thus entire groups are shrunken to 0.
- Examples:
  - SNPs/variants/CpG sites in the same genes.
  - Genes grouped into functional pathways
  - Indicator variables for the levels of a categorical variable

Group Lasso in Bioinformatics

BMC Bioinformatics

BioMed Central

Research

Open Access

Identifying main effects and epistatic interactions from large-scale SNP data via adaptive group Lasso

Can Yang*[1], Xiang Wan[1], Qiang Yang[2], Hong Xue[3] and Weichuan Yu*[1]

Bioinformatics, 34(2), 2018, 278–285
doi: 10.1093/bioinformatics/btx594
Advance Access Publication Date: 18 September 2017
Original Paper

OXFORD

Bioimage informatics

A novel SCCA approach via truncated ℓ_1-norm and truncated group lasso for brain imaging genetics

BMC Bioinformatics

BioMed Central

Methodology article

Open Access

Supervised group Lasso with applications to microarray data analysis

Shuangge Ma*[1], Xiao Song[2] and Jian Huang[3]

Group Lasso Example

```
## library(hgu95av2.db)

probe <- colnames(x)
get(probe[10], env=hgu95av2SYMBOL)

gene.ALL <- NULL
for (i in 1:length(probe)) {
     gene.ALL[i] <- get(probe[i], env=hgu95av2SYMBOL)
}
miss <- is.na(gene.ALL)
gene.ALL <- gene.ALL[!miss]
xg <- x[,!miss]
colnames(xg) <- gene.ALL

dim(xg)
length(gene.ALL)
length(unique(gene.ALL))
group.id <- as.numeric(factor(gene.ALL))
range(group.id)
```

Group Lasso Example

```
max(table(group.id))
xg[1:5, group.id==2809]
xg[1:5, group.id==7340]

oo <- order(group.id)
gid <- group.id[oo]
sx <- xg[,oo]

colnames(sx)[1:100]
gid[1:100]

## library(gglasso)

yk <- rep(1, length(y))
yk[y=="T"] <- -1
gg <- gglasso(sx, yk, group=gid, loss="logit")
```

Group Lasso Example

```
gg$lambda
dim(gg$beta)
gg$df
which(gg$beta[,100]!=0)
plot(gg)

t <- max(which(gid==700))
gt <- gglasso(sx[,1:t], yk, group=gid[1:t], loss="logit")
plot(gt)
which(gt$beta[,100]!=0)

##### Do not run ##########################################
# set.seed(123)
# cv2 <- cv.gglasso(sx, yk, group=gid, loss="logit")
# plot(cv2)

# coef(cv2$gglasso.fit, s=cv2$lambda.min)
# coef(cv2$gglasso.fit, s=cv2$lambda.1se)
```

Sparse Group Lasso

- Sparse Group Lasso(SGL) combines lasso and group lasso.
- SGL can perform group selection as well as individual selection.
- Suppose that the p predictors are divided into K groups of sizes q_k with corresponding coefficients $\boldsymbol{\beta}_k = (\beta_{k1}, \ldots, \beta_{kq_k})^{\mathrm{T}}$.
- The penalty function of SGL is

$$p_\lambda(\boldsymbol{\beta}) = \lambda \left(\alpha \sum_{k=1}^{K} \sum_{l=1}^{q_k} |\beta_{kl}| + (1-\alpha) \sum_{k=1}^{K} \sqrt{q_k} \|\boldsymbol{\beta}_k\|_2 \right),$$

  where $\|\boldsymbol{\beta}_k\|_2 = \left(\sum_{l=1}^{q_k} \beta_{kl}^2\right)^{1/2}$
- Sparse group lasso can be reduced to
  - lasso if $\alpha = 1$
  - group lasso if $\alpha = 0$

Sparse Group Lasso in Bioinformatics

Bioinformatics, 33(21), 2017, 3445–3453
doi: 10.1093/bioinformatics/btx427
Advance Access Publication Date: 4 July 2017
Original Paper

OXFORD

Systems biology

Linking metabolic network features to phenotypes using sparse group lasso

Satya Swarup Samal[1,*,†], Ovidiu Radulescu[2], Andreas Weber[3] and Holger Fröhlich[1,4,*]

BIOINFORMATICS **ORIGINAL PAPER** *Vol. 26 no. 19 2010, pages 2375–2382*
doi:10.1093/bioinformatics/btq448

Genome analysis Advance Access publication August 6, 2010

Association screening of common and rare genetic variants by penalized regression

Hua Zhou[1,2,*], Mary E. Sehl[3], Janet S. Sinsheimer[2,4,5] and Kenneth Lange[2,4,6]

[1]Department of Statistics, North Carolina State University, Raleigh, NC 27695-8203, [2]Departments of Human Genetics, [3]Departments of Medicine, [4]Departments of Biomathematics, [5]Departments of Biostatistics and [6]Departments of Statistics, University of California, Los Angeles, CA 90095, USA

Associate Editor: Alfonso Valencia

Sparse Group Lasso Example

```
## library(SGL)
miss <- is.na(ALL$age)
ya <- ALL$age[!miss]
ax <- sx[!miss,]

g1 <- SGL(list(x=ax, y=ya), index=gid, alpha=0, type="linear")
g2 <- SGL(list(x=ax, y=ya), index=gid, alpha=0.3, type="linear")
g3 <- SGL(list(x=ax, y=ya), index=gid, alpha=0.7, type="linear")
g4 <- SGL(list(x=ax, y=ya), index=gid, alpha=1, type="linear")

alp <- c(0, 0.3, 0.7, 1)
par(mfrow=c(2,2))
tt <- 10
for (i in 1:length(alp)) {
    g0 <- get(paste("g",i,sep=""))
    matplot(log(g0$lambdas)[1:tt], t(g0$beta[,1:tt]), type="l",
            lty=1, col="red", xlab="log.lambda", ylab="Coefficients",
            main=paste("alpha = ", alp[i]))
}
```

Sparse Group Lasso Example

```
DF <- matrix(0, length(g1$lambdas), 4)
rownames(DF) <- paste("lam", seq(length(g1$lambdas)), sep="")
colnames(DF) <- c("a0", "a0.3", "a0.7", "a1.0")
for (i in 1:4) {
    g0 <- get(paste("g",i,sep=""))
    DF[,i] <- apply(g0$beta, 2, function(t) sum(t!=0))
}
DF

tt <- 8
beta <- matrix(0, ncol(sx), 4)
rownames(beta) <- colnames(sx)
colnames(beta) <- c("a0", "a0.3", "a0.7", "a1.0")
for (i in 1:4) {
     g0 <- get(paste("g",i,sep=""))
     beta[,i] <- g0$beta[,tt]
}
ss <- apply(beta, 1, sum)
beta[ss!=0,]
```

Network-based Regularization in Bioinformatics

BIOINFORMATICS ORIGINAL PAPER Vol. 24 no. 9 2008, pages 1175–1182
doi:10.1093/bioinformatics/btn081

Systems biology

Network-constrained regularization and variable selection for analysis of genomic data

Caiyan Li and Hongzhe Li*
Department of Biostatistics and Epidemiology, University of Pennsylvania School of Medicine, Philadelphia, PA 19104, USA

Received on December 20, 2007; revised on February 17, 2008; accepted on February 27, 2008
Advance Access publication March 1, 2008
Associate Editor: Olga Troyanskaya

Research Article
Statistics in Medicine

Received 8 May 2012, Accepted 5 November 2012 Published online 5 December 2012 in Wiley Online Library
(wileyonlinelibrary.com) DOI: 10.1002/sim.5694

Network-based regularization for matched case-control analysis of high-dimensional DNA methylation data

BIOINFORMATICS ORIGINAL PAPER Vol. 28 no. 10 2012, pages 1368–1375
doi:10.1093/bioinformatics/bts145

Genetics and population analysis

Advance Access publication March 30, 2012

Penalized logistic regression for high-dimensional DNA methylation data with case-control studies

Hokeun Sun and Shuang Wang*

Limitation of Existing Regularization Methods

- In genomic studies, functionally similar genes are linked with each other via biological network;
  - Gene regulatory network
  - Signaling pathway
  - Metabolic pathway
  - Protein-protein interaction
- Network information has been accumulated over years.
- Biological network databases
  - Biocarta, HumanCyc, KEGG, Panther, PathBank, PharmGKB, Reactome and SMPDB
- Existing regulation methods are not able to utilize biological network information.
- Network-based regularization has been proposed to incorporate network information into genetic association studies.

Graph-constrained Penalty

- The graph-constrained penalty is

$$
\begin{aligned}
p_\lambda(\boldsymbol{\beta}) &= \lambda\alpha\|\boldsymbol{\beta}\|_1 + \lambda(1-\alpha)\boldsymbol{\beta}^{\mathrm{T}} L \boldsymbol{\beta} \\
&= \lambda\left(\alpha\sum_{j=1}^{p}|\beta_j| + (1-\alpha)\sum_{u=1}^{p}\sum_{u\sim v}\left(\frac{\beta_u}{\sqrt{d_u}} - \frac{\beta_v}{\sqrt{d_v}}\right)^2\right)
\end{aligned}
$$

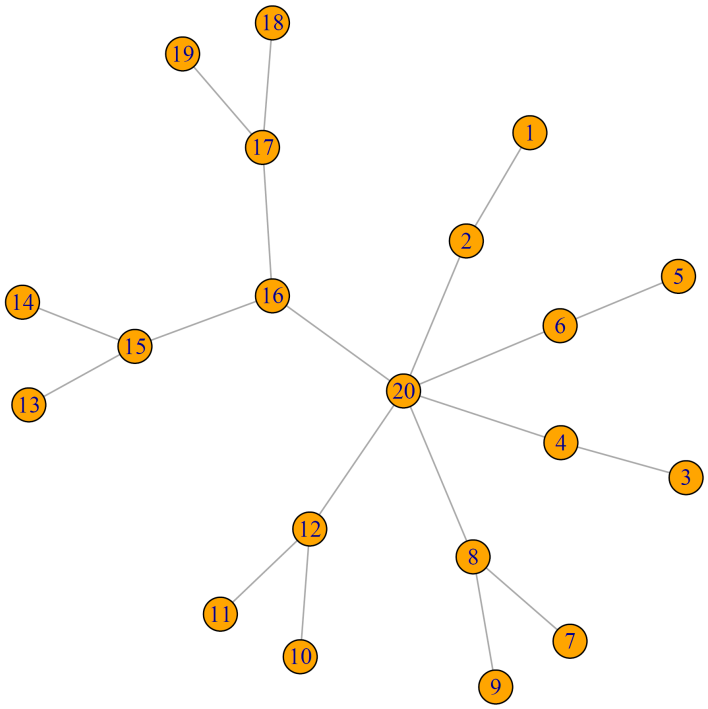
  and the p-dimensional Laplacian matrix $L = \{l_{uv}\}$ is

$$
l_{uv} = \begin{cases} 1 & \text{if} \quad u = v \text{ and } d_u \neq 0 \\ -(d_u d_v)^{-\frac{1}{2}} & \text{if} \quad u \text{ and } v \text{ are linked with each other} \\ 0 & \text{otherwise,} \end{cases}
$$

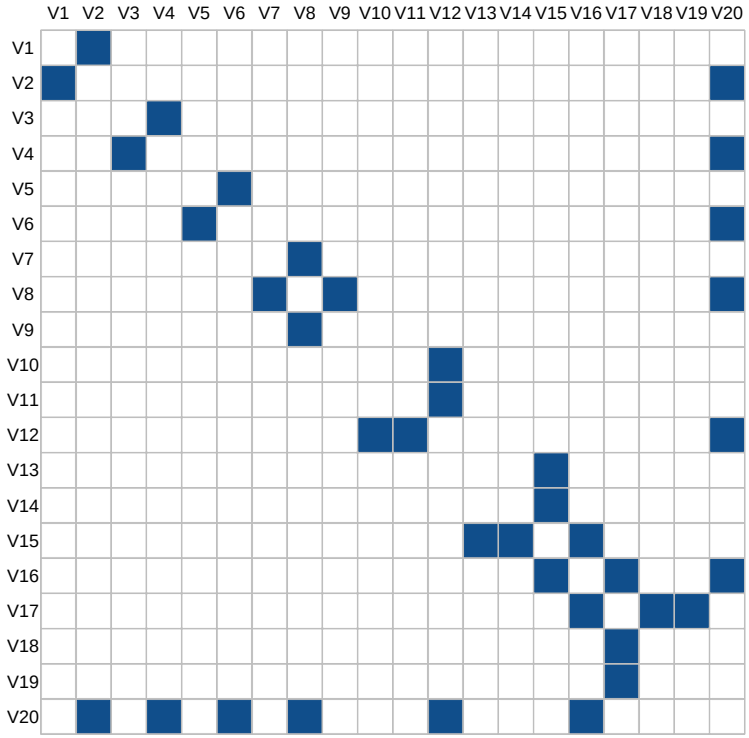
  where d_u is the total number of links of the u-th gene.
- Smoothness w.r.t linked genes (coefficients) is induced.
- Laplacain matrix represents graph structure among p genes.

Example of Laplacian Matrix

- 20 genes (nodes) have 19 genetic links (edges)
- 6 links: gene(V20), 3 links: 5 genes(V8, V12, V15, V16, V17), 2 links: 3 genes(V2, V4, V6), and 1 link: the other 11 genes.



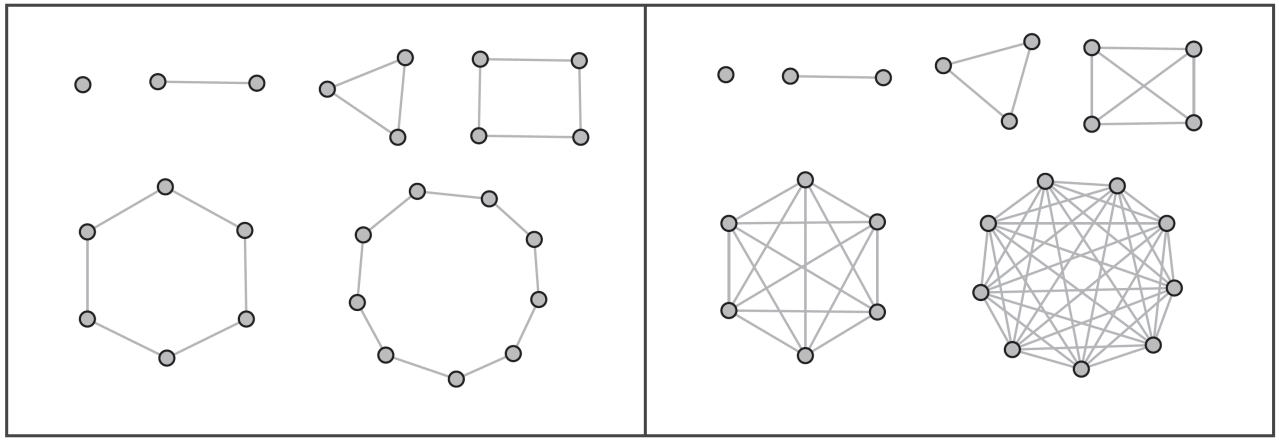
Network graph



Laplacian matrix

Network-based Regularization

- Network-based regularization was used for group structured data (Sun and Wang, 2012).
- Imaginary network structure for each gene
  - Ring network: adjacent sites/vartiants are only connected with each other
  - Fully connected (F.con) network: all sites/variants are fully connected with each other



Ring F.con

- Laplacian matrix has a block diagonal matrix

R package 'pclogit'

- **P**enalized **C**onditional **logi**stic regression model.

  https://github.com/statsun78/pclogit
- The pclogit estimates regression coefficients of the penalized network regression model along with λ values and computes selection probability of individual variables.
- Phenotype outcomes
  - Matched case-control outcomes
  - Unmatched case-control outcomes
- Network-based penalty function
  - Elastic-net penalty without a group or graph structure.
  - Both individual and group selection with a group structure
  - Both individual and group selection with a graph structure
- No covariates are allowed.

Network-based Regularization Example

```
## install.packages('devtools')
## library(devtools)
## install_github("statsun78/pclogit")
library(pclogit)
?pclogit

yn <- rep(1, length(y))
yn[y=="T"] <- 0
oo <- order(group.id)
gsize <- as.numeric(table(group.id[oo]))

gp1 <- pclogit(sx, yn, alpha=0.5, group=gsize, type="ring")
gp2 <- pclogit(sx, yn, alpha=0.5, group=gsize, type="fcon")
cbind(gp1$df, gp2$df)

par(mfrow=c(1,2))
matplot(log(gp1$lambda), t(gp1$beta), type="l", lty=1, col="red",
        xlab="log.lambda", ylab="Coefficients", main="Ring network")
matplot(log(gp2$lambda), t(gp2$beta), type="l", lty=1, col="red",
        xlab="log.lambda", ylab="Coefficients", main="F.con network")
```

Network-based Regularization Example

```
tt <- 30

beta <- matrix(0, ncol(sx), 2)
rownames(beta) <- colnames(sx)
colnames(beta) <- c("Ring", "F.con")
beta[,1] <- gp1$beta[,tt]
beta[,2] <- gp2$beta[,tt]
ss <- apply(beta, 1, sum)
beta[ss!=0,]

set.seed(1234)
u <- 1000
sxx <- sx[ ,1:sum(gsize[1:u])]

sp1 <- sel.pclogit(sxx, yn, alpha=0.5, group=gsize[1:u], type="ring")
sp1$maxsel[1:50,]

##### Do not run ##############################################
# sp1 <- sel.pclogit(sx, yn, alpha=0.5, group=gsize, type="ring")
# sp2 <- sel.pclogit(sx, yn, alpha=0.5, group=gsize, type="fcon")
```

Network-based Regularization Example

```
## library(graphite)

kegg <- pathways("hsapiens", "kegg")
kegg.nodes <- rep(0, length(kegg))
for (i in 1:length(kegg)) {
    kegg.nodes[i] <- length(nodes(kegg[[i]]))
}
kegg.nodes

kn <- 89
kegg[[kn]]
ksym <- convertIdentifiers(kegg[[kn]], "SYMBOL")
genes.kn <-  gsub("SYMBOL:", "", nodes(ksym))
genes.kn

sum(!is.na(match(unique(genes.kn), unique(gene.ALL))))

wm <- which(!is.na(match(genes.kn, unique(gene.ALL))))
genes.kn[wm]
```

Network-based Regularization Example

```
edges(ksym)[, c(2,4)]

links <- edges(ksym)[, c(2,4)]
adj <- matrix(0, length(wm), length(wm))
for (i in 1:nrow(links)) {
    m <- match(links[i,], genes.kn[wm])
    if (sum(is.na(m))==0) {
        adj[m[1], m[2]] <- adj[m[2], m[1]] <- 1
    }
}
dim(adj)

## library(igraph)
ig <- graph.adjacency(adjmatrix= adj , mode="undirected")
plot(ig, vertex.label=genes.kn[wm], vertex.size=8, edge.width=1,
     vertex.color="orange", vertex.label.color="black",
     vertex.label.cex=0.8, edge.color="black", vertex.label.dist=0.3,
     layout=layout_with_kk)
## detach(package:igraph)
```

Network-based Regularization Example

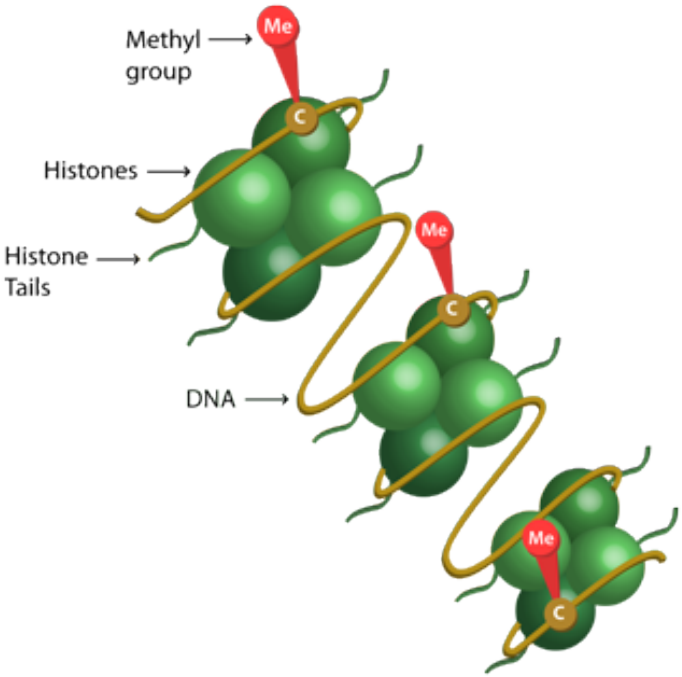
```
kg <- genes.kn[wm]
kg

to <- 0
for (i in 1:length(kg)) to[i] <- sum(kg[i]==gene.ALL)
data.frame(gene=kg, num=to)

newx <- matrix(0, nrow(x), length(kg))
colnames(newx) <- kg
rownames(newx) <- rownames(x)
for (i in 1:length(kg)) {
    w <- which(kg[i]==gene.ALL)
    newx[,i] <- prcomp(sx[,w])$x[,"PC1"]
}

set.seed(1010)
gs <- sel.pclogit(newx, yn, alpha=0.5, group=adj, K=200)
gs$maxsel
```

1. Statistical Variable Selection and Cross-validation
2. Analysis of High-dimensional Genomic Data I
3. Analysis of High-dimensional Genomic Data II

4. Analysis of High-dimensional DNA Methylation Data

DNA Methylation

- DNA methylation is the addition of a methyl group to the 5' position of cytosine in the context of a CpG dinucleotide.
- Adding methyl groups change the appearance and structure of DNA.



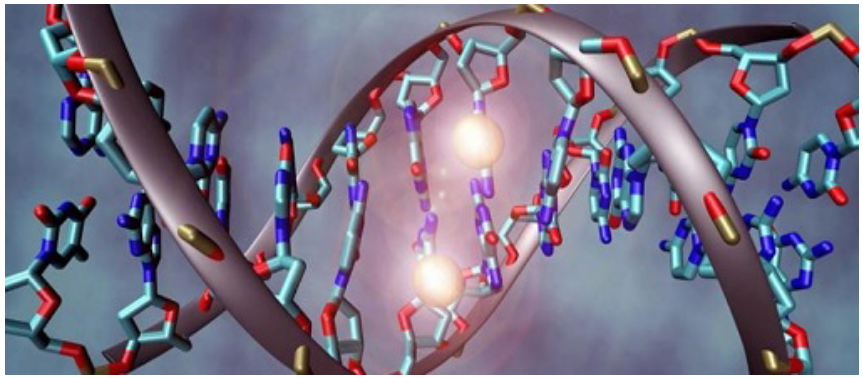


Illustration of a DNA molecule that is methylated at the cytosines.

High-dimensional DNA Methylation Data

- CpG (Cytosine-phosphate-Guanine) rich regions are frequently hyper-methylated in cancer tissues, but not in normal tissues.
- DNA methylation data have recently been generated from high-throughput DNA methylation platforms
  - Illumina Infinium HumanMethylation450 BeadChip array.
  - Illumina Infinium MethylationEPIC BeadChip array.

Design Markers | **Process BeadChip** | Analyze Data

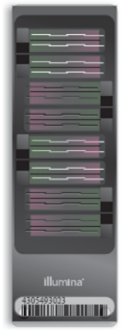
illumina®

Infinium® MethylationEPIC BeadChip

Affordable methylome analysis meets cutting edge content.

Highlights

- **Unique Combination of Coding Region and Enhancer-Wide Coverage**
  Over 850,000 methylation sites per sample at single-nucleotide resolution
- **High Assay Reproducibility**
  > 98% reproducibility for technical replicates
- **Simple Workflow**
  PCR-free protocol with the powerful Infinium HD Assay
- **Compatible with FFPE Samples**
  Protocol available for methylation studies on FFPE samples



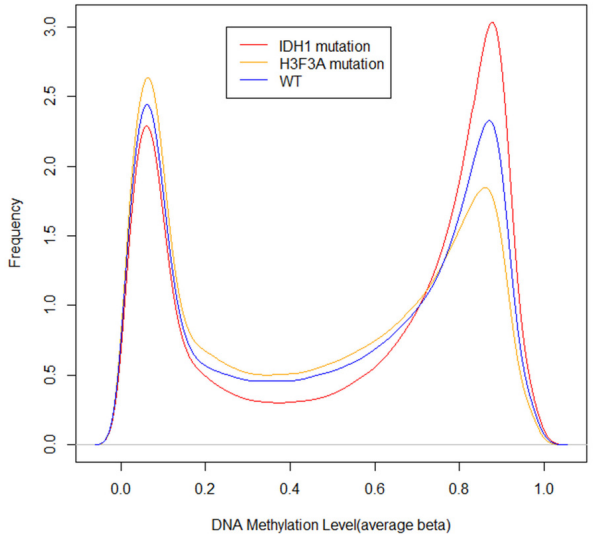
Methylation β-values

- The methylation array data of the CpG sites are a sequence of β-values.
- For each locus (CpG site)

$$\beta = \frac{\max(M, 0)}{\max(U, 0) + \max(M, 0) + 100},$$

where U is the fluorescent signal from an unmethylated allele and M is that from a methylated allele.

- This β-value ranges continuously from 0 (unmethylated) to 1 (completely methylated)
- The methylation levels are enriched at 0 and 1.



Analysis of High-dimensional DNA Methylation Data

- Research goal
  - Identify differentially methylated CpG sites and genes between cancer cases and healthy controls over entire genomes.
  - (Matched) case-control association study with high-dimensional DNA data
- Statistical problem
  - Variable selection for high-dimensional data
  - Methylation values of multiple CpG sites within the same genes are highly correlated with each other.
- Regularization methods have been proposed.
  - Sun and Wang (2012, *Bioinformatics*)
  - Sun and Wang (2013, *Stat. Med.*)
  - Sun *et al.* (2017, *Bioinformatics*)
  - Choi *et al.* (2018, *J. Bioinform. Comput. Biol.*)
  - Kim and Sun (2019, *BMC Bioinformatics*)
  - Kim *et al.* (2020, *J. Stat. Comput. Simul.*)

Ovarian Cancer Study from 27K DNA Methylation Data

- The data is available at the NCBI Gene Expression Omnibus.
- Investigated differentially methylated CpG sites between ovarian cancer cases and healthy controls.
  - Teschendorff, A. *et al.* (2010, *Genome Ressearch*)
  - Wang, S. (2011, *Genetic Epidemiology*)
  - Sun and Wang (2012, *Bioinformatics*)
  - Huang, H. *et al.* (2013, *BMC Bioinformatics*)
  - Chen, Y. *et al.* (2014, *Genetic Epidemiology*)
- Data from Illumina Infinium HumanMethylation27K Beadchip.
- After data processing, we ended up with 20,461 CpG sites from 12,770 genes.
- 12,770 genes have from 1 to 22 CpG sites.
- 152 healthy controls and 123 ovarian cancer cases.

Ovarian Cancer Data

```
data(ovarian27k)
str(ovarian27k)
names(ovarian27k)

head(ovarian27k$genename)
tail(ovarian27k$genename)

gsize <- ovarian27k$group
c(length(gsize), sum(gsize))
table(gsize)

ww <- which.max(gsize)
cc <- cumsum(gsize)[(ww-1):ww]
ovarian27k$genename[(cc[1]+1):cc[2], c(2,3)]

y <- ovarian27k$y
y
length(y)
table(y)
```

Ovarian Cancer Data

```
x <- ovarian27k$x
dim(x)
summary(as.numeric(x))
hist(x, nclass=50, col="orange")

set.seed(25)
par(mfrow=c(5,5))
s <- sample(1:ncol(x), 25)
for (i in 1:25) {
  si <- s[i]
  hist(x[,si], nclass=20, col="orange", xlab="",
       main=paste("CpG site = ", s[i]))
}

xm <- log2(x/(1-x))
summary(as.numeric(xm))
par(mfrow=c(1,2))
hist(x, nclass=100, col="orange",
     main=expression(paste(beta,"-values")))
hist(xm, nclass=100, col="orange", main="M-values")
```

Univariate Analysis

```
pval.t <- apply(xm, 2, function(t) t.test(t[y==0], t[y==1])$p.val)
pval.w <- apply(xm, 2, function(t) wilcox.test(t[y==0], t[y==1])$p.val)

ap <- 0.05
oo.t <- order(pval.t)
bt <- p.adjust(pval.t, method="bonferroni")
ft <- p.adjust(pval.t, method="BH")
st <- max(sum(bt < ap), sum(ft < ap))
ot <- oo.t[1:st]
gt <- ovarian27k$genename[ot, c(2,3)]
data.frame(gt, p.value=pval.t[ot], Bonferroni=bt[ot], fdr.BH=ft[ot])

oo.w <- order(pval.w)
bw <- p.adjust(pval.w, method="bonferroni")
fw <- p.adjust(pval.w, method="BH")
sw <- max(sum(bw < ap), sum(fw < ap))
ow <- oo.w[1:sw]
gw <- ovarian27k$genename[ow, c(2,3)]
data.frame(gw, p.value=pval.w[ow], Bonferroni=bw[ow], fdr.BH=fw[ow])
```

Univariate Analysis

```
cc <- rep("gray", length(pval.t))
cc[ft < ap] <- "darkblue"; cc[bt < ap] <- "red"
plot(-log10(pval.t), type="p", pch=20, col=cc, xlab="CpG sites",
     ylab=expression(paste("-log"[10],"(P)")),
     main="Independent two sample T-test")
abline(h=-log10(max(pval.t[bt < ap])),col="red", lty=2, lwd=1.5)
abline(h=-log10(max(pval.t[ft < ap])),col="darkblue", lty=3, lwd=1.5)
legend("topleft", c("5% Bonferroni","5% fdr"), lty=c(2,3), lwd=2,
        col=c("red", "darkblue"))

cw <- rep("gray", length(pval.w))
cw[fw < ap] <- "darkblue"; cw[bw < ap] <- "red"
plot(-log10(pval.w), type="p", pch=20, col=cw, xlab="CpG sites",
     ylab=expression(paste("-log"[10],"(P)")),
     main="Wilcoxon Rank Sum Test")
abline(h=-log10(max(pval.w[bw < ap])),col="red", lty=2, lwd=1.5)
abline(h=-log10(max(pval.w[fw < ap])),col="darkblue", lty=3, lwd=1.5)
legend("topleft", c("5% Bonferroni","5% fdr"), lty=c(2,3), lwd=2,
        col=c("red", "darkblue"))
```

Lasso: Ovarian Cancer Data

```
## library(glmnet)
set.seed(1111)

alpha <- 1
g1 <- cv.glmnet(xm, y, alpha=alpha, family="binomial", nfolds=5)
plot(g1)

fit11 <- coef(g1, s="lambda.min")[-1]
fit12 <- coef(g1, s="lambda.1se")[-1]
c(sum(as.matrix(fit11)!=0), sum(as.matrix(fit12)!=0))

w11 <- which(as.matrix(fit11)!= 0)
w12 <- which(as.matrix(fit12)!= 0)
gn11 <- ovarian27k$genename[w11, c(2,3)]
gn12 <- ovarian27k$genename[w12, c(2,3)]

data.frame(gn11, beta=fit11[w11])
data.frame(gn12, beta=fit12[w12])
```

Elastic-net: Ovarian Cancer Data

```
par(mfrow=c(2,1))
plot(abs(fit11), type="h", col="darkgreen", xlab="CpG sites",
     ylab=expression(paste("|",hat(beta),"|")), ylim=c(0, 1.1),
     main="Lasso using lambda.min")
plot(abs(fit12), type="h", col="darkgreen", xlab="CpG sites",
     ylab=expression(paste("|",hat(beta),"|")), ylim=c(0, 1.1),
     main="Lasso using lambda.1se")

set.seed(1234)
alpha <- 0.5

g2 <- cv.glmnet(xm, y, alpha=alpha, family="binomial", nfolds=5)
plot(g2)

fit21 <- coef(g2, s="lambda.min")[-1]
fit22 <- coef(g2, s="lambda.1se")[-1]
c(sum(as.matrix(fit21)!=0), sum(as.matrix(fit22)!=0))
```

Elastic-net: Ovarian Cancer Data

```
w21 <- which(as.matrix(fit21)!= 0)
w22 <- which(as.matrix(fit22)!= 0)

gn21 <- ovarian27k$genename[w21, c(2,3)]
gn22 <- ovarian27k$genename[w22, c(2,3)]

data.frame(gn21, beta=fit21[w21])
data.frame(gn22, beta=fit22[w22])

par(mfrow=c(2,1))
plot(abs(fit21), type="h", col="darkgreen", xlab="CpG sites",
     ylab=expression(paste("|",hat(beta),"|")), ylim=c(0, 1.1),
     main="Elastic-net using lambda.min")
plot(abs(fit22), type="h", col="darkgreen", xlab="CpG sites",
     ylab=expression(paste("|",hat(beta),"|")), ylim=c(0, 1.1),
     main="Elastic-net using lambda.1se")
```

Group Lasso: Ovarian Cancer Data

```
## library(gglasso)
set.seed(1111)

grp <- rep(1:length(gsize), gsize)
ry <- y
ry[y==0] <- -1

##### Do not run ########################################
# g3 <- cv.gglasso(xm, ry, group=grp, loss="logit", nfolds=5)

ur1 <- url("https://github.com/statsun78/kogo/raw/master/download/
            g3g4g5.Rdata")
load(ur1)

fit31 <- coef(g3, s = "lambda.min")[-1]
fit32 <- coef(g3, s = "lambda.1se")[-1]
c(sum(as.matrix(fit31)!=0), sum(as.matrix(fit32)!=0))
```

Group Lasso: Ovarian Cancer Data

```
w31 <- which(as.matrix(fit31)!= 0)
w32 <- which(as.matrix(fit32)!= 0)
gn31 <- ovarian27k$genename[w31, c(2,3)]
gn32 <- ovarian27k$genename[w32, c(2,3)]

data.frame(gn31, beta=fit31[w31])
data.frame(gn32, beta=fit32[w32])

par(mfrow=c(2,1))
plot(abs(fit31), type="h", col="darkgreen", xlab="CpG sites",
     ylab=expression(paste("|",hat(beta),"|")), ylim=c(0, 1.1),
     main="Group Lasso using lambda.min")
plot(abs(fit32), type="h", col="darkgreen", xlab="CpG sites",
     ylab=expression(paste("|",hat(beta),"|")), ylim=c(0, 1.1),
     main="Group Lasso using lambda.1se")
```

Network-based Regularization: Ovarian Cancer Data

```
## library(pclogit)
set.seed(1212)

##### Do not run ########################################
# g4 <- sel.pclogit(xm, y, alpha=0.1, group=gsize, type="ring")
# g5 <- sel.pclogit(xm, y, alpha=0.1, group=gsize, type="fcon")

top <- 10
gn41 <- ovarian27k$genename[g4$maxsel[1:top,1], c(2,3)]
gn51 <- ovarian27k$genename[g5$maxsel[1:top,1], c(2,3)]

data.frame(gn41, SP=g4$maxsel[1:top,2])
data.frame(gn51, SP=g5$maxsel[1:top,2])

thr <- 0.8
t4 <- g4$maxsel[,2] > thr
t5 <- g5$maxsel[,2] > thr
```

Network-based Regularization: Ovarian Cancer Data

```
gn42 <- ovarian27k$genename[g4$maxsel[t4,1], c(2,3)]
gn52 <- ovarian27k$genename[g5$maxsel[t5,1], c(2,3)]

data.frame(gn42, SP=g4$maxsel[t4,2])
data.frame(gn52, SP=g5$maxsel[t5,2])

sp4 <- apply(g4$beta, 1, max)
sp5 <- apply(g5$beta, 1, max)

par(mfrow=c(2,1))
plot(sp4, type="h", col="darkgreen", xlab="CpG sites",
     ylab="Selection probability", ylim=c(0, 1.1),
     main="Network-based regularization using a ring network")
plot(sp5, type="h", col="darkgreen", xlab="CpG sites",
     ylab="Selection probability", ylim=c(0, 1.1),
     main="Network-based regularization using a f.con network")
```

Network-based Regularization: Ovarian Cancer Data

```
## library(graphite)

kegg <- pathways("hsapiens", "kegg")
reac <- pathways("hsapiens", "reactome")

##### Do not run ########################################
# gene.k <- suppressMessages(convertIdentifiers(kegg, "SYMBOL"))
# gene.r <- suppressMessages(convertIdentifiers(reac, "SYMBOL"))

ur2 <- url("https://github.com/statsun78/kogo/raw/master/download/
            pathways.Rdata")
load(ur2)

gene.kegg <- as.list(rep(NA, length(kegg)))
for (i in 1:length(kegg)) {
    knodes <- nodes(gene.k[[i]])
    gene.kegg[[i]] <- gsub("SYMBOL:", "", knodes)
}
```

Network-based Regularization: Ovarian Cancer Data

```
kgene <- sort(unique(unlist(gene.kegg)))
length(kgene)

gene.reac <- as.list(rep(NA, length(reac)))
for (i in 1:length(reac)) {
    rnodes <- nodes(gene.r[[i]])
    gene.reac[[i]] <- gsub("SYMBOL:", "", rnodes)
}
rgene <- sort(unique(unlist(gene.reac)))
length(rgene)

sum(!is.na(match(kgene, rgene)))

ogene <- unique(ovarian27k$genename[,3])
length(ogene)

sum(!is.na(match(ogene, kgene)))
sum(!is.na(match(ogene, rgene)))
```

Network-based Regularization: Ovarian Cancer Data

```
mk <- match(kgene, ogene)
rk <- match(rgene, ogene)

og <- c(mk[!is.na(mk)], rk[!is.na(rk)])
fgene <- ogene[sort(unique(og))]
length(fgene)

adjm <- matrix(0, length(fgene), length(fgene))

for (j in 1:length(kegg)) {
    ek1 <- edges(gene.k[[j]])[, 2]
    ek2 <- edges(gene.k[[j]])[, 4]
    wk1 <- match(ek1, fgene)
    wk2 <- match(ek2, fgene)
    k12 <- !is.na(wk1) & !is.na(wk2)
    if (sum(k12) > 0) {
            adjm[cbind(wk1[k12], wk2[k12])] <- 1
            adjm[cbind(wk2[k12], wk1[k12])] <- 1
    }
}
```

Network-based Regularization: Ovarian Cancer Data

```
for (j in 1:length(reac)) {
    er1 <- edges(gene.r[[j]])[, 2]
    er2 <- edges(gene.r[[j]])[, 4]
    wr1 <- match(er1, fgene)
    wr2 <- match(er2, fgene)
    r12 <- !is.na(wr1) & !is.na(wr2)
    if (sum(r12) > 0) {
            adjm[cbind(wr1[r12], wr2[r12])] <- 1
            adjm[cbind(wr2[r12], wr1[r12])] <- 1
    }
}

diag(adjm) <- 0
dd <- apply(adjm, 1, sum)
dd0 <- dd > 0

genes <- fgene[dd0]
length(genes)
```

Network-based Regularization: Ovarian Cancer Data

```
adj <- adjm[dd0, dd0]
dim(adj)
sum(adj)/2

dg <- apply(adj, 1, sum)
summary(dg)
data.frame(gene=genes, degree=dg)

newx <- matrix(0, nrow(x), length(genes))
colnames(newx) <- genes
for (i in 1:length(genes)) {
    ww <- which(genes[i]==ovarian27k$genename[,3])
newx[,i] <- prcomp(xm[ ,ww])$x[,"PC1"]
}

##### Do not run ########################################
# g6 <- sel.pclogit(newx, y, alpha=0.1, group=adj, K=200)

g6$maxsel[g6$maxsel[,2] > 0.8, ]
```

Network-based Regularization: Ovarian Cancer Data

```
adj0 <- matrix(0, length(ogene), length(ogene))
mm <- match(genes, ogene)
adj0[mm, mm] <- adj

newx0 <- matrix(0, nrow(x), length(ogene))
colnames(newx0) <- ogene
for (i in 1:length(ogene)) {
    wx <- which(ogene[i]==ovarian27k$genename[,3])
    newx0[ ,i] <- prcomp(xm[ ,wx])$x[,"PC1"]
}

##### Do not run ##########################################
# g7 <- sel.pclogit(newx0, y, alpha=0.1, group=adj0, K=200)

g7$maxsel[g7$maxsel[,2] > 0.8, ]
```